

5.

$$E_{\text{out}}(G) = P(G(x) \neq y)$$

$$= P\left(\text{sign}\left(\sum_{t=1}^{2M+1} g_t(x)\right) \neq y\right)$$

To let $\text{sign}\left(\sum_{t=1}^{2M+1} g_t(x)\right) \neq y$, we need more than half of classifiers make error

Let $Y_t = 1$ if $g_t(x) \neq y$, $Y_t = 0$ otherwise.

$$\Rightarrow P(Y_t = 1) = e_t$$

Let $S = \sum_{t=1}^{2M+1} Y_t$, the total number of classifiers make errors

$$\Rightarrow E[S] = \sum_{t=1}^{2M+1} e_t$$

$$\Rightarrow E_{\text{out}}(G) = P\left(\text{sign}\left(\sum_{t=1}^{2M+1} g_t(X)\right) \neq y\right)$$

$$= P(S \geq M+1)$$

According to Markov inequality,

$$P(S \geq M+1) \leq \frac{E(S)}{a} = \frac{\sum_{t=1}^{2M+1} e_t}{M+1} \quad \#$$

6.

According to the lecture,

$$u_n^{(t+1)} = u_n^{(t)} \exp(-\alpha_n y_n g_t(\underline{x}_n))$$

$$\Rightarrow U_{t+1} = \sum_{n=1}^N u_n^{(t+1)} = \sum_{n=1}^N u_n^{(t)} \exp(-\alpha_n y_n g_t(\underline{x}_n))$$

$$= \sum_{n: y_n = g_t(\underline{x}_n)} u_n^{(t)} \exp(-\alpha_t) + \sum_{n: y_n \neq g_t(\underline{x}_n)} u_n^{(t)} \exp(\alpha_t)$$

Since the weight of correctly / incorrectly rate is $(1 - \epsilon_t) / \epsilon_t$, we can rewrite U_{t+1} :

$$U_{t+1} = \exp(-\alpha_t)(1 - \epsilon_t) U_t + \exp(\alpha_t) \epsilon_t U_t$$

$$\begin{aligned}
\frac{U_{t+1}}{U_t} &= \exp(-\alpha_t)(1-E_t) + \exp(\alpha_t)E_t \\
&= \sqrt{\frac{E_t}{1-E_t}}(1-E_t) + \sqrt{\frac{1-E_t}{E_t}}(E_t) \\
&= 2\sqrt{E_t(1-E_t)} \quad \#
\end{aligned}$$

7.

According to the lecture, we need to find optimal y for

$$\min \frac{1}{N} \sum_{n=1}^N ((y_n - s_n) - y g_1(\underline{x}_n))^2$$

$$\Rightarrow y = \frac{\sum_{n=1}^N g_1(\underline{x}_n)(y_n - s_n)}{\sum_{n=1}^N g_1^2(\underline{x}_n)}$$

For the convenience, assume $s_1 = s_2 = \dots = s_N = 0$ initially.

$$\Rightarrow y = \frac{\sum_{n=1}^N g_1(\underline{x}_n)(y_n)}{\sum_{n=1}^N g_1^2(\underline{x}_n)}$$

Given that $g_1(\underline{x}_n)$ is a linear regression trained to fit y_n , according to the previous lecture, let $g_1(\underline{x}_n) = \underline{w}_1^T \underline{x}_n$

the predictions $\hat{\underline{y}} = X \underline{w}_1$, where X is the feature Matrix, is the projection of $\underline{y}$ onto the column space of X ,

and hence $\underline{y} - \hat{\underline{y}} \perp \hat{\underline{y}}$

$$\Rightarrow \sum_{n=1}^N \hat{y}_n (y_n - \hat{y}_n) = 0 \Rightarrow \sum_{n=1}^N y_n \hat{y}_n = \sum_{n=1}^N \hat{y}_n^2$$

$$\Rightarrow \alpha_1 = y = \frac{\sum_{n=1}^N g_1(\underline{x}_n)(y_n)}{\sum_{n=1}^N g_1^2(\underline{x}_n)}$$

$$= \frac{\sum_{n=1}^N y_n \hat{y}_n}{\sum_{n=1}^N \hat{y}_n^2} = 1 \quad \#$$

8.

$$S_n^{(t+1)} = S_n^{(t)} + \alpha_t g_t(\underline{x}_n),$$

$$\alpha_t = \underset{y}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^N ((y_n - S_n) - y g_t(\underline{x}_n))^2$$

$$\Rightarrow \alpha_t = \frac{\sum_{n=1}^N (y_n - S_n) g_t(\underline{x}_n)}{\sum_{n=1}^N g_t^2(\underline{x}_n)}$$

substitute

$$\Rightarrow \sum_{n=1}^N (y_n - S_n^{(t+1)}) g_t(\underline{x}_n)$$

$$= \sum_{n=1}^N (y_n - S_n^{(t)} - \alpha_t g_t(\underline{x}_n)) g_t(\underline{x}_n)$$

$$= \sum_{n=1}^N (y_n - S_n^{(t)}) g_t(\underline{x}_n) - \sum_{n=1}^N \alpha_t g_t^2(\underline{x}_n) = 0$$

#

9.

Inductive proof :

①

all weights $w_{i,j}^{(2)}$ are initially set to 0.5 ,
so $w_{i,j}^{(1)} = w_{i,j+1}^{(1)}$ for all i and $1 \leq j \leq d^{(1)}$
is established

②

Assume $w_{i,j}^{(1)} = w_{i,j+1}^{(1)}$ for all i and

$1 \leq j \leq d^{(1)}$ at iteration t , then

$$s_j^{(1)} = \sum_{i=0}^{d^{(0)}} w_{i,j}^{(1)} x_i^{(0)}$$

which are the same for all $s_j^{(1)}$, $1 \leq j \leq d^{(1)}$

and hence $x_j = \tanh(s_j)$ are all the same

for $1 \leq j \leq d^{(1)}$, too

$$\begin{aligned}
\Rightarrow \delta_j^{(1)} &= \frac{\partial e_n}{\partial \delta_j^{(1)}} = \sum_{k=1}^{d^{(2)}} \frac{\partial e_n}{\partial s_k^{(2)}} \frac{\partial s_k^{(2)}}{\partial x_j^{(1)}} \frac{\partial x_j^{(1)}}{\partial \delta_j^{(1)}} \\
&= \frac{\partial e_n}{\partial s_1^{(2)}} \frac{\partial s_1^{(2)}}{\partial x_j^{(1)}} \frac{\partial x_j^{(1)}}{\partial \delta_j^{(1)}} \quad \left(\text{only 1 hidden layer} \right) \\
&\quad \downarrow^{(2)} = 1 \\
&= -2(y_n - s_1^{(2)}) w_{j1}^{(2)} \tanh'(\delta_j^{(1)})
\end{aligned}$$

which are the same for all $\delta_j^{(1)}$, $1 \leq j \leq d^{(1)}$

and hence after the update

$$w_{i,j}^{(1)} = w_{i,j}^{(1)} - \eta x_i^{(0)} \delta_j^{(1)} = w_{i,j+1}^{(1)} - \eta x_i^{(0)} \delta_{j+1}^{(1)} = w_{i,j+1}^{(1)}$$

for all i and $1 \leq j \leq d^{(1)}$

③

By Induction, $w_{i,j}^{(1)} = w_{i,j+1}^{(1)}$ for all i and $1 \leq j \leq d^{(1)}$ holds for all t $\#$

Problem 10

$E_{in}(g_t)$ doesn't decrease as t increases but oscillates, I think it may be because that in AdaBoost algorithm, it focuses mainly on the $E_{in}^u(g_t)$, so it neither guarantees nor cares what E_{in} will be. Besides, ϵ_t slightly increases as t increases. I think it may be because that AdaBoost iteratively focuses on the examples that the current weak learner misclassifies, as the number of iterations increases, there are some "hard" examples which are difficult to be classified correctly, and their weights keep increasing, hence result in the overall larger ϵ_t .



Figure 1:

```
class DecisionStump:
    def __init__(self):
        self.feature_index = None
        self.threshold = None
        self.polarity = 1

    def fit(self, X, y, weights):
        n_samples, n_features = X.shape
        min_error = float('inf')

        for feature in range(n_features):
            thresholds = np.unique(X[:, feature])
            for threshold in thresholds:
                for polarity in [-1, 1]:
                    predictions = np.ones(n_samples)
                    if polarity == 1:
                        predictions[X[:, feature] < threshold] = -1
                    else:
                        predictions[X[:, feature] >= threshold] = -1

                    # take example weights into account
                    error = np.sum(weights[y != predictions])

                    if error < min_error:
                        min_error = error
                        self.feature_index = feature
                        self.threshold = threshold
                        self.polarity = polarity

    def predict(self, X):
        n_samples = X.shape[0]
```

Figure 2:

```

def predict(self, X):
    n_samples = X.shape[0]

    predictions = np.ones(n_samples)
    if self.polarity == 1:
        predictions[X[:, self.feature_index] < self.threshold] = -1
    else:
        predictions[X[:, self.feature_index] >= self.threshold] = -1
    return predictions

def adaboost(X_train, y_train, X_test, y_test, T=500):
    # initialize u_1 = [1/N, 1/N, ..., 1/N]
    n_samples = X_train.shape[0]
    weights = np.ones(n_samples) / n_samples

    alpha_list = []
    classifiers = []
    Ein_gt_list, epsilon_t_list = [], [] # problem 10
    Ein_Gt_list, Eout_Gt_list = [], [] # problem 11
    Ut_list = [1] # problem 12

    for t in range(T):
        # train g_t
        stump = DecisionStump()
        stump.fit(X_train, y_train, weights)
        predictions = stump.predict(X_train)

        # compute Ein(g_t) & epsilon_t
        misclassified = (predictions != y_train).astype(int)

```

Figure 3:

```

        # compute Ein(g_t) & epsilon_t
        misclassified = (predictions != y_train).astype(int)

        Ein_gt = np.sum(misclassified) / n_samples
        epsilon_t = np.dot(weights, misclassified) / np.sum(weights)

        Ein_gt_list.append(Ein_gt)
        epsilon_t_list.append(epsilon_t)

        # compute alpha_t and update weights
        alpha_t = 0.5 * np.log((1 - epsilon_t) / epsilon_t)
        weights *= np.exp(-alpha_t * y_train * predictions)
        # normalize weight, which isn't needed
        # weights /= np.sum(weights)

        # compute Ut_list
        Ut_list.append(np.sum(weights))

        alpha_list.append(alpha_t)
        classifiers.append(stump)

        # compute Ein(G_t)
        combined_predictions = np.sign(
            np.sum([alpha * clf.predict(X_train) for alpha, clf in zip(alpha_list, classifiers)], axis=0)
        )
        Ein_Gt = np.mean(combined_predictions != y_train)
        Ein_Gt_list.append(Ein_Gt)

        # compute Eout(G_t)
        combined_predictions = np.sign(
            np.sum([alpha * clf.predict(X_test) for alpha, clf in zip(alpha_list, classifiers)], axis=0)
        )
        Eout_Gt = np.mean(combined_predictions != y_test)
        Eout_Gt_list.append(Eout_Gt)

    return Ein_gt_list, epsilon_t_list, Ein_Gt_list, Eout_Gt_list, Ut_list[:-1]

```

Figure 4:

```
def plot(list_1, list_2, list_1_name, list_2_name, num_of_prob, times):

    t = list(range(1, times + 1))

    plt.figure(figsize=(10, 6))

    plt.plot(t, list_1, label=f'{list_1_name}', color='blue', linestyle='-', marker='o')
    plt.plot(t, list_2, label=f'{list_2_name}', color='red', linestyle='--', marker='x')

    plt.xlabel('t')
    plt.ylabel('Values')
    plt.title(f'Plot {list_1_name} and {list_2_name} as Function of t')
    plt.legend()
    plt.savefig(f'./hw7_{num_of_prob}_{times}.png')
```

Figure 5:

Problem 11

$E_{in}(G_t)$ decreases as t increases, which is trivial since as the number of weak learners increases, G_t becomes more powerful and can fit the training data better. But it seems that $E_{out}(G_t)$ doesn't have an obvious decrease, moreover, $E_{in}(G_t)$ and $E_{out}(G_t)$ is already close to each other at the beginning, so it may be because that the decision stump model, or AdaBoost with initial few weak learners is already powerful enough, and as the number of weak learners increases, it becomes too powerful and starts overfitting.

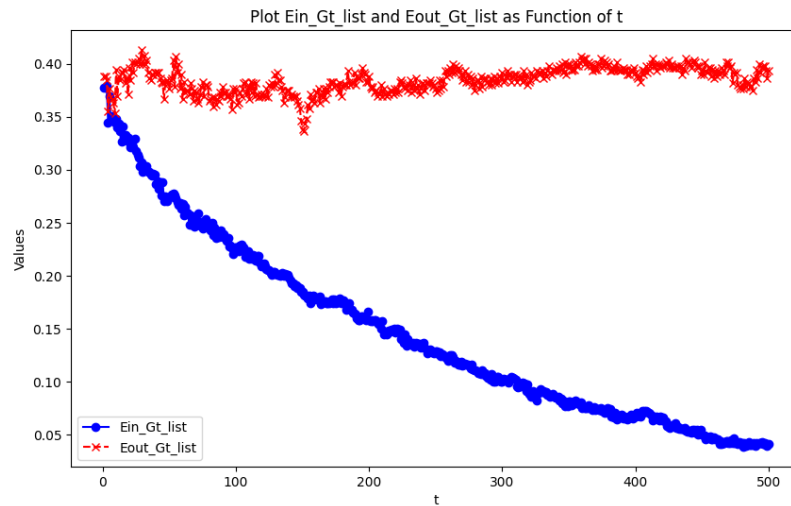


Figure 6:

Problem 12

Both $E_{in}(G_t)$ and U_t decreases as t increase, which match the lecture teaching : AdaBoost decreases

$$U_t = \sum_{n=1}^N u_n^{(t)} \text{ and thus somewhat minimizes } \sum_{n=1}^N u_n^{(T+1)} = \frac{1}{N} \sum_{n=1}^N \widehat{err}_{\text{ADA}}(s, y_n).$$

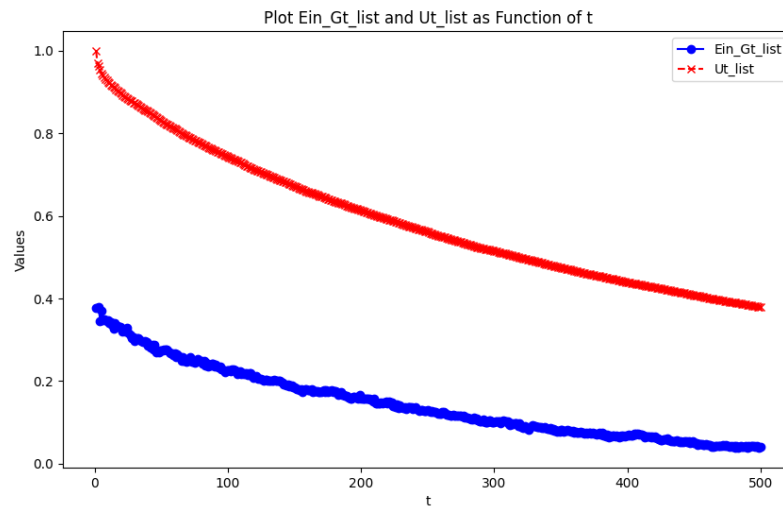


Figure 7: